TRICOLOR

Distributed HPC Cluster

Design · Implementation · Benchmarking

Liu Neptali Restrepo Sanabria
CPSC-375 · Trinity College Hartford · April 2026

OVERVIEW

Why *Tricolor*

Tricolor

"Three colours" is the name the Venezuelan flag receives.
"Tri" plays with the first three letters of Trinity College

Node Names

Node1 = Araguaney
Node2 = Turpial
Node3 = Orquidea

OVERVIEW

What was built

3 Compute Nodes

Lenovo ThinkStation P340
i7-10700 · 8c/16t · 16 GB each

Head Node

Laptop as controller
Job launch & SSH management

Private Network

Netgear switch · 1 GbE
Static IPs 192.168.1.x

Shared Storage

NFS from laptop → all nodes
/home/liu/cluster everywhere

IMPLEMENTATION

Software Stack & Configuration

SOFTWARE STACK

OS	Ubuntu Server 22.04 LTS
MPI	OpenMPI 4.1.6
BLAS	OpenBLAS 0.3.26
BLAS (alt)	ATLAS 3.10.3
Compiler	GCC + GFortran
Benchmark	HPL 2.3 (source)

KEY CONFIGURATION STEPS

1. Fresh Ubuntu Server on all nodes
2. Static IPs (192.168.1.101–103)
3. Passwordless SSH via Ed25519 keys
4. NFS share: laptop to all nodes
5. HPL compiled from source vs OpenBLAS
6. soft NFS mount with 3s timeout

Two-Phase Benchmarking Strategy

PHASE 1 · Strong Scaling

Fixed parameters, vary nodes

N=40,000 NB=232 4 proc/node

→ Does adding hardware help?
→ Exposes communication cost

1 node → 2 nodes → 3 nodes

PHASE 2 · Parameter Tuning

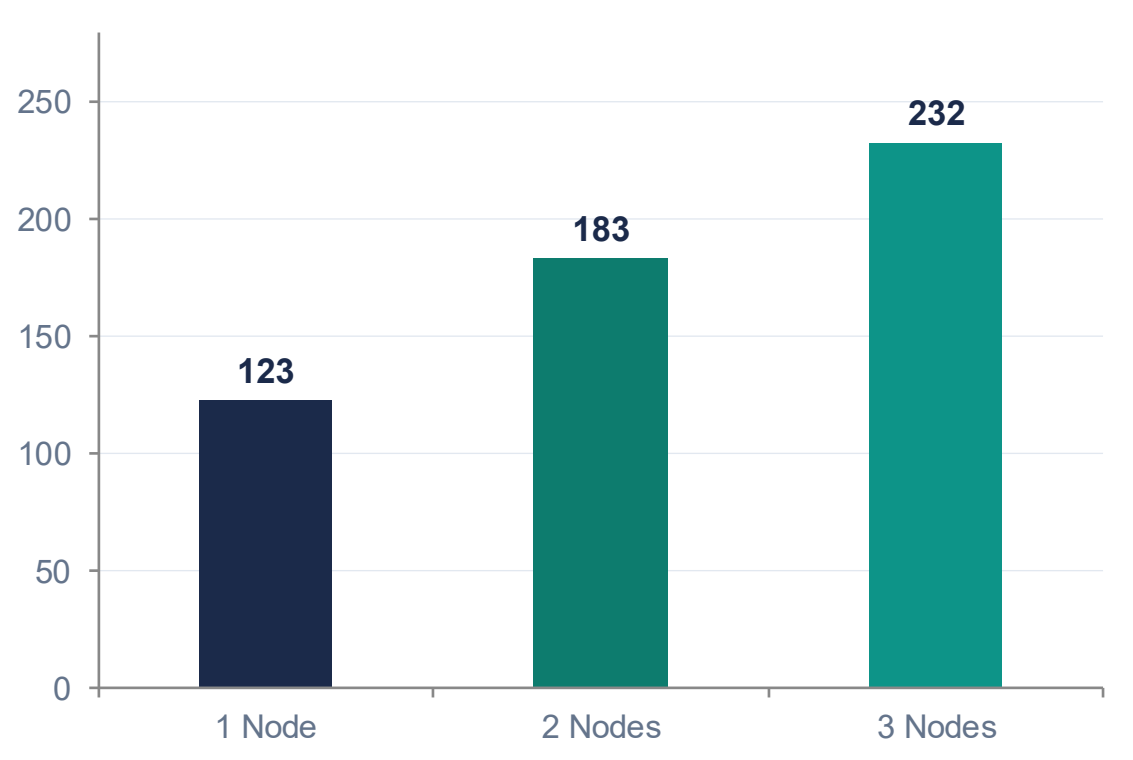
Fixed 3 nodes, vary parameters

NB: 232 → 256 → 384
Proc: 12 → 24 → 48
N: 40k → 52k → 60k

→ How sensitive is performance to tuning choices?

Different Block Sizes for testing cache management

Strong Scaling Results



1.50×

Speedup 1→2 nodes

74.8% efficiency

1.90×

Speedup 1→3 nodes

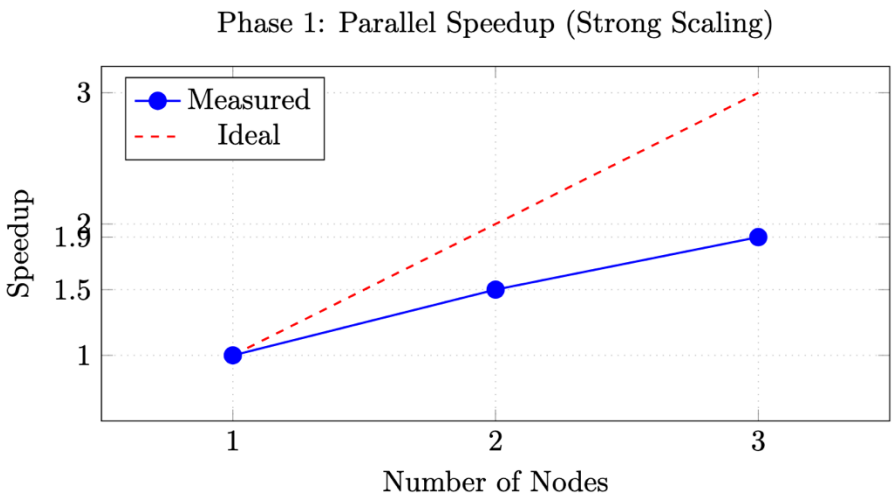
63.2% efficiency

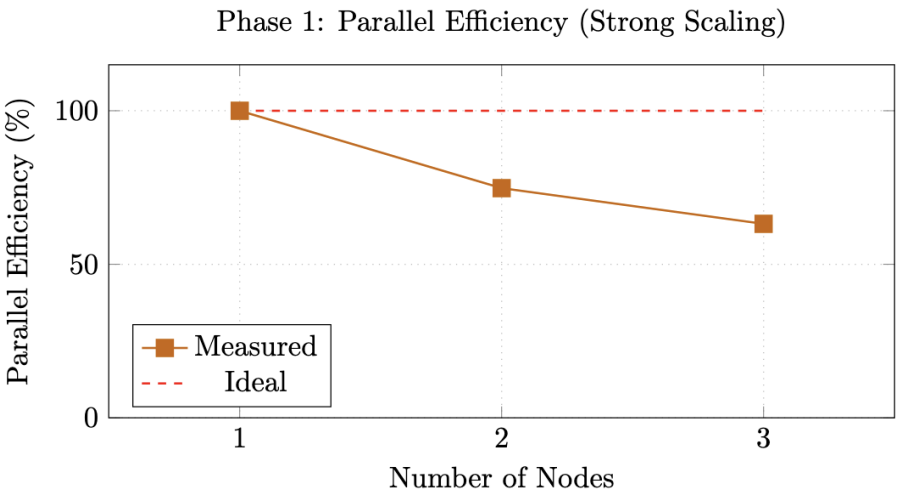
232 GFLOPS

Peak GFLOPS

+97.6% vs 1 node

Strong Scaling Results





Parameter Tuning: All Runs

Phase	Proc.	N	NB	GFLOPS
1	4	40,000	232	122.50
1	8	40,000	232	183.27
1	12	40,000	232	232.30
2	24	52,000	256	234.42
2	48	60,000	**256**	**218.54 ↓**
2	48	60,000	**384**	**242.01 ★**

Over-provisioning backfires

48 proc + NB=256 scored BELOW the 12-process baseline.
More processes ≠ more speed.

NB=384 is the key lever

Larger blocks → better cache utilisation → fewer MPI syncs per unit of computation.
+10.7% gain over NB=256.

Performance Analysis

AMDAHL'S LAW

Speedup is bounded by the serial fraction

$$S(N) = \frac{1}{f + \frac{1-f}{N}}$$

S = total speedup · N = nodes · s = serial fraction

With N=3 nodes giving 1.90× speedup, fitting Amdahl's law implies a serial fraction s ≈ 29%. Even with infinite nodes, max speedup ≈ 3.4×.

Performance Analysis

The model is optimistic

Comm. overhead

MPI sync cost grows with problem size; Amdahl assumes it's free.

Memory Contention

Processes per node share the same bus and L3 cache

Load Imbalance

One slow node stalls the whole collective

OBSERVED EFFICIENCY

Each added node costs more than it gives

74.8%
@ 2 nodes

63.2%
@ 3 nodes

Linear scaling would mean N nodes deliver an N× speedup at 100% efficiency. The drop from 74.8% to 63.2% is the signature of a non-trivial serial portion plus growing coordination cost.

CHALLENGES FACED

Challenges & Solutions

SSH / MPI

Mac MPI incompatible with node OpenMPI — mpirun hung indefinitely

Launch mpirun from a node, not the Mac. Redistribute SSH keys under correct username (liu)

NFS

Sessions freezing randomly; hard mount semantics blocked indefinitely on failure

Remounted with soft NFS + 3s timeout in /etc/fstab. Prevents hangs on network hiccup.

HPL Build

Compilation failed: rogue mpicc in Make.auxil rule and missing -I include path for hpl.h

Patched two lines in makes/Make.auxil. Required reading raw Makefile internals.

Tuning

48-process run scored below 12-process baseline; more processes degraded performance

Identified NB block size as the bottleneck. NB=384 recovered +10.7% over NB=256.

Learning Outcomes

01

Distributed computing works — but communication costs

242 GFLOPS peak on commodity hardware. Efficiency 100% → 63% as nodes grow, bottlenecked by 1 GbE.

02

More processes is not more performance

48-process run at NB=256 fell below the 12-process baseline. Process count must balance communication overhead.

03

Block size is the most impactful single tuning lever

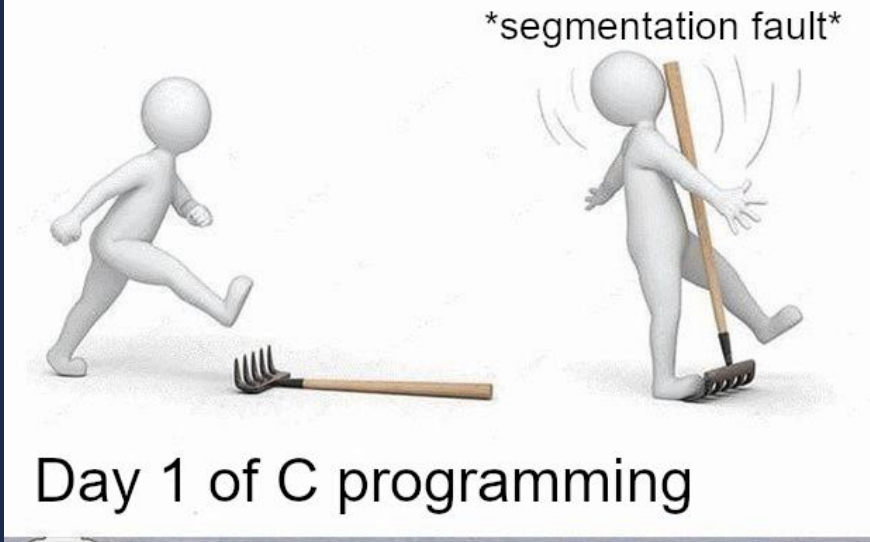
NB 256→384 yielded +10.7% with identical hardware. Cache utilisation matters as much as parallelism.

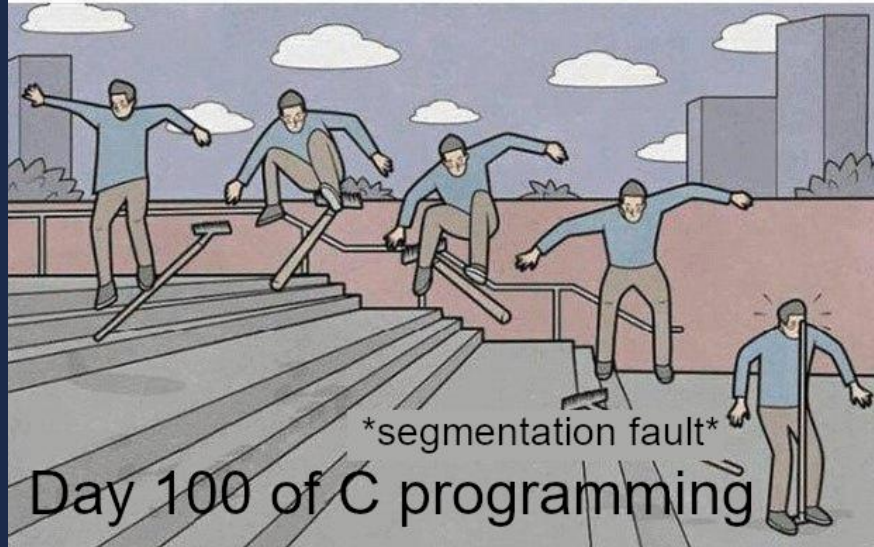
04

Two-phase methodology gives rigorous system understanding

Strong scaling reveals communication limits; parameter tuning finds the operating point. Real HPC workflow.

THAT’S IT!


segmentation fault
Day 1 of C programming


segmentation fault
Day 100 of C programming